

Android #11 Application Lifecycle

Table of Contents

Android #11 Application Lifecycle.....	1
Table of Contents	1
Pengantar	1
Lifecycle.....	2
Memahami 7 Callback Utama.....	2
Perbedaan Logika: Prosedural dan Event-Driven	2
Mengapa Implementasi Lifecycle Krusial	2
Pemahaman Mengenai Siklus Hidup Aplikasi melalui Callback pada Android Kotlin.....	3
Referensi	5
Tugas Pendalaman Materi (Skenario Pengujian Logger Application).....	5

Pengantar

Dalam program konsol sederhana (seperti C++ atau Python dasar), kode Anda berjalan secara berurutan dari atas ke bawah (main function). Namun di Android, OS-lah yang memegang kendali penuh atas sumber daya perangkat.

Aplikasi tidak tahu kapan pengguna akan menerima telepon, kapan baterai akan habis, atau kapan pengguna tiba-tiba memutar layar. Daripada aplikasi Anda terus-menerus bertanya ke sistem (polling), Android yang akan "menghubungi" aplikasi Anda melalui callback saat sesuatu terjadi.

Bayangkan jika tidak ada callback dan aplikasi Anda harus menggunakan loop while(true) untuk mengecek apakah layar masih terlihat atau tidak. Hal ini akan menguras baterai dalam hitungan menit.

Dengan callback:

- Sistem : "Hei, pengguna baru saja menekan tombol Home, saya panggil onStop() ya. Silakan matikan koneksi internet atau animasi kamu agar hemat baterai."
- Aplikasi : "Siap, laksanakan!"

Lifecycle

Activity Lifecycle adalah kumpulan transisi status yang dialami oleh sebuah Activity, mulai dari pertama kali dibuat hingga dihancurkan oleh sistem. Transisi ini dikelola melalui fungsi-fungsi khusus yang disebut Callback Methods.

Memahami 7 Callback Utama

Setiap fase dalam siklus hidup memiliki tugas spesifik yang harus dipahami oleh pengembang:

- `onCreate()`: Dipanggil saat Activity pertama kali dibuat.
- `onStart()`: Saat Activity mulai terlihat di layar, tetapi belum bisa berinteraksi dengan pengguna.
- `onResume()`: Saat Activity berada di baris terdepan (foreground) dan pengguna bisa mulai mengklik tombol atau mengetik.
- `onPause()`: Saat Activity masih terlihat sedikit tetapi kehilangan fokus (misal: muncul dialog permission atau pop-up transparan).
- `onStop()`: Saat Activity sudah tidak terlihat sama sekali (misal: pengguna menekan tombol Home atau membuka Activity lain).
- `onRestart()`: Dipanggil hanya jika Activity kembali dari status `onStop` menuju `onStart`.
- `onDestroy()`: Saat Activity benar-benar dihapus dari memori (misal: menekan tombol Back atau memanggil fungsi `finish()`).

Perbedaan Logika: Prosedural dan Event-Driven

Fitur	Prosedur	Event-Driven
Kendali	Kode yang menentukan alur	Sistem (OS) yang menentukan alur
Eksekusi	Berurutan dari baris satu ke akhir	Melompat ke fungsi tertentu saat ada kejadian (event)
Fungsi Utama	<code>main()</code>	<code>onCreate()</code> , <code>onStart()</code> , dll

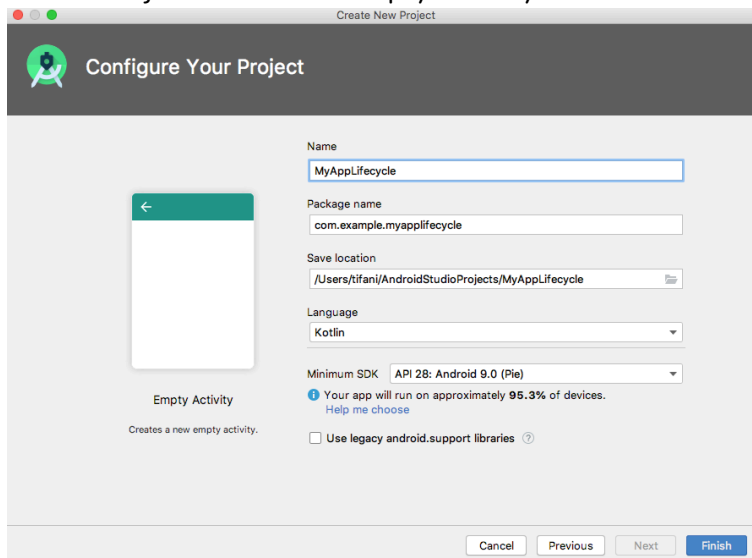
Mengapa Implementasi Lifecycle Krusial

- Crash Aplikasi
- Memory Leak
- Pemborosan Baterai
- Kehilangan Data

Pemahaman Mengenai Siklus Hidup Aplikasi melalui Callback pada Android Kotlin

Untuk memperjelas pemahaman mengenai materi ini, kita menggunakan Logger Application untuk memonitoring siklus hidup aplikasi.

1. Buat projec android seperti biasa dengan Buka Android Studio -> Start a New Android Studio Project -> Next -> Empty Activity -> Next



Finish.

2. Tambahkan dua widget pada activity_main.xml seperti berikut ini:

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.appcompat.widget.LinearLayoutCompat
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:gravity="center"
    tools:context=".MainActivity">

    <TextView
        android:id="@+id/textStatus"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Pantau Logcat untuk melihat Lifecycle"
        android:textSize="18sp" />

    <Button
        android:id="@+id/btnFinish"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="20dp"
        android:text="Hentikan Activity (Finish)" />
```

```
</androidx.appcompat.widget.LinearLayoutCompat>
```

3. Tambahkan kode pada file MainActivity.kt untuk monitoring Logcat seperti berikut.

```
package com.example.applifecycle

import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle
import android.util.Log
import android.widget.Button

class MainActivity : AppCompatActivity() {
    private val TAG = "LifecycleCheck`"

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val btnFinish : Button = findViewById(R.id.btnFinish)

        btnFinish.setOnClickListener {
            // Memanggil finish() akan memicu onPause -> onStop -> onDestroy
            finish()
        }

        Log.d(TAG, "onCreate: Activity Dibuat")
    }

    override fun onStart() {
        super.onStart()
        Log.d(TAG, "onStart: Activity Terlihat")
    }

    override fun onResume() {
        super.onResume()
        Log.d(TAG, "onResume: Activity Interaktif (Running)")
    }

    override fun onPause() {
        super.onPause()
        Log.d(TAG, "onPause: Activity Dijeda (Sebagian tertutup)")
    }

    override fun onStop() {
        super.onStop()
        Log.d(TAG, "onStop: Activity Berhenti (Tidak terlihat)")
    }

    override fun onDestroy() {
        super.onDestroy()
        Log.d(TAG, "onDestroy: Activity Dihancurkan")
    }

    override fun onRestart() {
        super.onRestart()
        Log.d(TAG, "onRestart: Activity Dijalankan Kembali")
    }
}
```

4. Lakukan skenario pengujian pada tugas pendalaman materi

Referensi

<https://developer.android.com/>

Tugas Pendalaman Materi (Skenario Pengujian Logger Application)

Skenario	Urutan Callback	Penjelasan
Aplikasi dibuka pertama kali	...	...
Menekan tombol home	...	...
Membuka apps kembali dari recents	...	...
Menekan tombol back	...	...
Rotasi layar	...	...
Tambahkan skenario lainnya ...	...	...